Lab Workshop Challenges Solutions

Systems Analysis & Design — Hands-On Application (Labs 01, 02, & 03)

LAB 01 FURPS+ Drone Delivery Framework

Dimension	Requirement Description	Measurable Target / Specification
Functionality	Autonomous point-to-point dispatch & drop-off	Fly to target hospital GPS coords with ±0.5m landing precision.
Usability	Emergency medical staff payload retrieval	One-touch OTP door unlock on arrival completed in < 15 seconds.
Reliability	Fail-safe emergency landing & connection lost	Deploy parachute automatically if communication lost for > 30s.
Performance	Cold-chain thermal drug delivery speed	Deliver 2kg payload up to 15km in < 20 min maintaining 2°C–8°C.
Supportability	Modular battery & telemetry swapping	Field swap battery unit and reboot systems in < 3 minutes.
+ Security	Encrypted command link signal	AES-256 encrypted telemetry link to prevent mid-air hijacking.
+ Compliance	Civil Aviation Authority airspace regulatory adherence	Real-time automatic geofencing anti-collision adherence (ADS-B).

LAB 02 Peer-to-Peer Car-Share 2D Story Map & MVP Skeleton

Walking Skeleton (MVP): Highlighted green cards form the minimum viable path required to execute a end-to-end trip booking.

1. User Onboarding

- **1.1 Driver Verification (MVP)**
- 1.2 Background Identity Check
- 1.3 User Loyalty Rating

2. Vehicle Search

- **2.1 GPS Radius Search (MVP)**
- 2.2 Advanced Model Filter
- 2.3 EV Battery Range Filter

3. Unlock & Drive

- **3.1 BLE Digital Key Unlock (MVP)**
- 3.2 Pre-trip Damage Scanner
- 3.3 In-cabin Climate Control

4. Return & Pay

- **4.1 Card Auto-Billing (MVP)**
- 4.2 Toll Fare Auto-Split
- 4.3 Cleaning Fee Dispute

Lab Workshop Challenges Solutions (Cont.)

Domain-Driven Design (DDD) Aggregate Implementation

LAB 03 University Enrollment Domain Model (DDD)

Modeling the `CourseEnrollment` Aggregate Root. Enforces business invariant boundaries (maximum seat capacity & prerequisite completion) and emits domain events upon successful state changes.

```
// Aggregate Root: CourseEnrollment public class CourseEnrollment { private final CourseId
courseId; private final int maxCapacity = 30; private Set<StudentId> enrolledStudents = new
HashSet<>(); private List<DomainEvent> domainEvents = new ArrayList<>(); // Business Method
maintaining Invariants public void enrollStudent(Student student, PrerequisiteChecker checker) {
// Invariant Rule 1: Capacity Limit Check if (enrolledStudents.size() >= maxCapacity) { throw new
CourseFullException("Enrollment cap of 30 reached."); } // Invariant Rule 2: Prerequisite
Validation if (!checker.hasCompletedPrerequisites(student.getId(), this.courseId)) { throw new
PrerequisiteNotMetException("Missing prerequisite courses."); } // State Transition
enrolledStudents.add(student.getId()); // Emit Domain Event this.domainEvents.add(new
StudentEnrolledEvent(student.getId(), this.courseId)); } }
```

Emitted Domain Events & Downstream Actions

- **`StudentEnrolledEvent`:** Triggered when a student successfully registers. Listened to by the *Billing Service* to automatically calculate and generate tuition fees.
- **`CourseCapacityReachedEvent`:** Triggered when `enrolledStudents.size() == 30`. Listened to by the *Registrar Service* to lock further registration requests.